

Equazione del Calore 2D e Riordinamento nelle Fattorizzazioni Cholesky

Giosuè Aiello

1 Impostazione del problema

L'obiettivo del progetto è la risoluzione numerica dell'equazione del calore in regime stazionario in due dimensioni spaziali. Il dominio di calcolo è il quadrato unitario $\Omega = [0, 1]^2$, e si cerca la funzione di temperatura $u : \Omega \rightarrow \mathbb{R}$ che soddisfa l'equazione ellittica:

$$\kappa \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) + f(x, y) = 0, \quad (x, y) \in \Omega, \quad (1)$$

con condizioni al contorno di Dirichlet omogenee sul bordo $\partial\Omega$:

$$u(x, y) = u_0(x, y) \equiv 0, \quad (x, y) \in \partial\Omega.$$

I parametri fisici del problema sono: coefficiente di diffusività termica $\kappa = 0.01$ e termine sorgente $f(x, y) = e^{-10(x^2+y^2)}$, che modella una sorgente di calore concentrata in prossimità dell'origine con decadimento gaussiano.

Discretizzazione alle differenze finite

Si introduce una griglia cartesiana uniforme di $(N + 2)^2$ punti (x_i, y_j) con $i, j = 0, \dots, N + 1$ e passo $h = 1/(N + 1)$, dove $x_i = i \cdot h$ e $y_j = j \cdot h$. I punti di bordo hanno temperatura fissata a zero e non contribuiscono come incognite. Le N^2 incognite del problema sono i valori $u_{i,j} \approx u(x_i, y_j)$ nei punti *interni* ($1 \leq i, j \leq N$).

Approssimando le derivate seconde con lo stencil a cinque punti, l'equazione (1) valutata nel punto interno (x_i, y_j) diventa:

$$\frac{\kappa}{h^2} (u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - 4u_{i,j}) + f(x_i, y_j) = 0. \quad (2)$$

Se un vicino cade sul bordo, il suo valore (zero) viene incorporato nel termine noto senza aggiungere incognite. L'insieme delle equazioni (2) costituisce il sistema lineare $Au = b$, dove $A \in \mathbb{R}^{N^2 \times N^2}$ è sparsa (al più 5 elementi non nulli per riga), simmetrica e definita negativa.

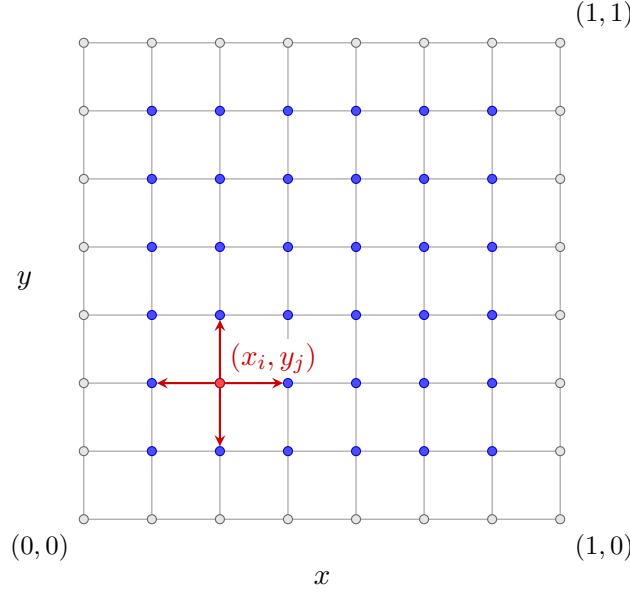


Figura 1: Punti esterni ed interni del dominio, segnati rispettivamente in grigio ed in blu, e punti adiacenti, evidenziati in rosso per un punto di esempio (x_i, y_j) .

Fattorizzazione di Cholesky e fill-in

Poiché $-A$ è simmetrica definita positiva (SPD), si può calcolare la fattorizzazione di Cholesky $-A = LL^T$ con L triangolare inferiore sparsa. Tuttavia, durante la fattorizzazione si generano elementi non nulli in posizioni che erano zero in $-A$: questo fenomeno è detto *fill-in* e dipende fortemente dall'ordine con cui le incognite sono enumerate.

Con l'ordinamento naturale per righe, la larghezza di banda di A è $\Theta(N)$ e il fill-in di L è $O(N^3)$. La tecnica di *nested dissection* riduce il fill-in a $O(N^2 \log N)$, con un risparmio sostanziale per valori grandi di N .

Grafo di adiacenza e nested dissection

La struttura di A è codificata dal *grafo di adiacenza* $G = (V, E)$: ogni nodo $v \in V$ corrisponde a una variabile $u_{i,j}$ e ogni arco $(u, v) \in E$ indica che $A_{uv} \neq 0$. Per la griglia regolare, due nodi sono collegati se e solo se sono vicini orizzontalmente o verticalmente:

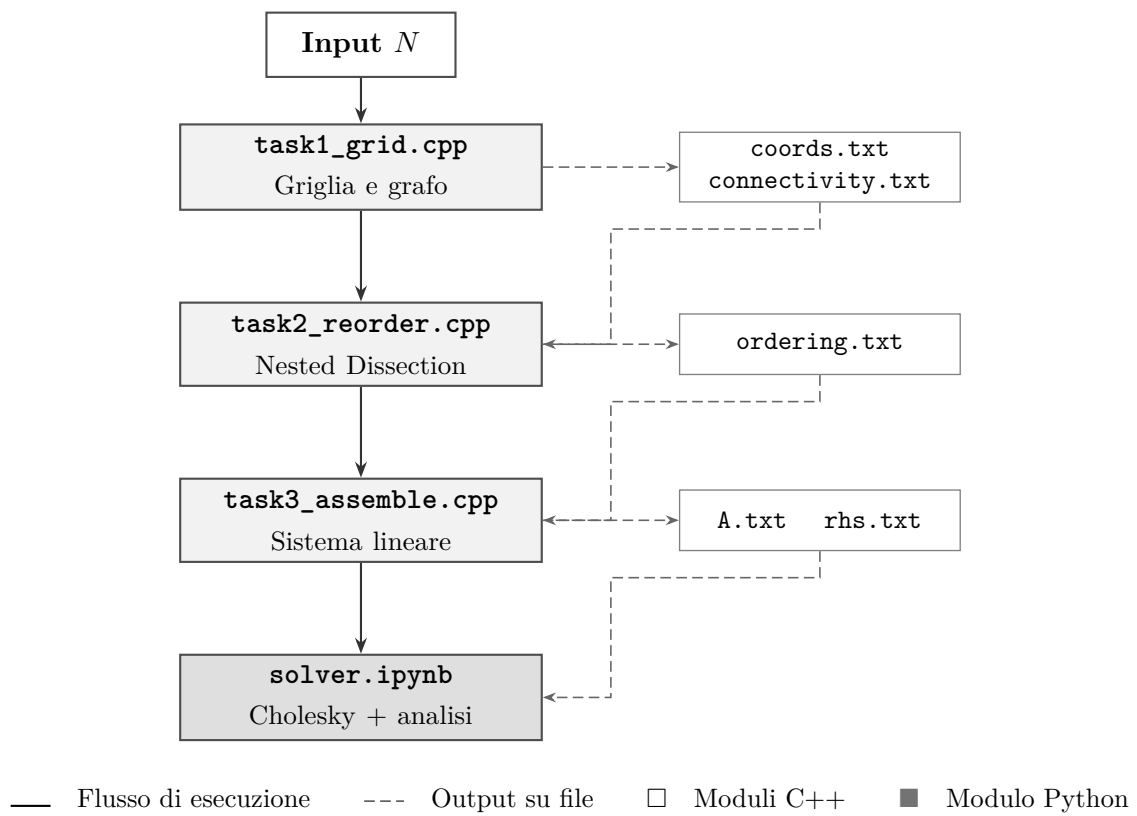
$$(x_i, y_j) \leftrightarrow (x_k, y_l) \iff (i = k, |j - l| = 1) \text{ oppure } (j = l, |i - k| = 1).$$

La *nested dissection* individua ricorsivamente un separatore geometrico V_S di cardinalità ridotta che divide V in due insiemi bilanciati V_1 e V_2 senza archi diretti tra loro. L'ordinamento risultante elenca prima V_1 , poi V_2 , infine V_S , in modo che il separatore appaia in fondo alla matrice, limitando la propagazione del fill-in. Per la griglia regolare il separatore ottimale è la colonna mediana: scelto $\hat{x}$ mediano, si pone $V_1 = \{u_{i,j} \mid x_i < \hat{x}\}$, $V_2 = \{u_{i,j} \mid x_i > \hat{x}\}$, $V_S = \{u_{i,j} \mid x_i = \hat{x}\}$. Il procedimento viene applicato ricorsivamente a V_1 e V_2 , alternando il taglio lungo x e y .

2 Architettura del progetto

Il progetto è strutturato in quattro moduli. I tre moduli in C++ comunicano tra loro attraverso file di testo con formato fisso che fungono da interfaccia verificabile indipendentemente; il modulo Python li legge come punto di ingresso per la fase numerica finale.

Il diagramma seguente illustra il flusso di elaborazione dall'input N alla soluzione e all'analisi comparativa:



Ogni modulo ha un ruolo preciso all'interno della pipeline:

task1_grid.cpp*C++*

Riceve N ; genera i N^2 nodi interni con indici e coordinate (`coords.txt`) e la lista degli archi del grafo di adiacenza (`connectivity.txt`).

task2_reorder.cpp*C++*

Legge il grafo e calcola la permutazione nested dissection ricorsiva: ad ogni livello individua il separatore geometrico mediano (alternando asse x e y) e lo posiziona in coda. Salva la corrispondenza in `ordering.txt`.

task3_assemble.cpp*C++*

Assembla la matrice sparsa A (stencil a 5 punti: $-4\kappa/h^2$ in diagonale, $+\kappa/h^2$ fuori diagonale) e il termine noto b . Supporta ordinamento naturale e nested dissection. Produce `A.txt` e `rhs.txt`.

solver.ipynb*Python*

Carica `A.txt` e `rhs.txt` in formato CSC; fattorizza $-A = LL^T$ con `sksparse.cholmod`; risolve con `spsolve_triangular`. Confronta tempi per $N = 32 \dots 1024$; produce spy plot e grafici della soluzione $u(x, y)$.

Le strutture dati fondamentali sono: liste di adiacenza (`std::vector<std::vector<int>>`) per il grafo nei moduli C++, e matrici sparse in formato CSC (`scipy.sparse.csc_matrix`) nel modulo Python.

3 Specifiche dei moduli

I quattro moduli vengono di seguito descritti con livello di dettaglio crescente: per ciascuno si indicano input, output, funzionalità richieste, strutture dati e complessità attesa.

Modulo 1 — `task1_grid.cpp`: generazione della griglia e del grafo

Il Modulo 1 è responsabile della creazione della struttura geometrica del dominio e del relativo grafo di adiacenza. Per realizzarlo, ho deciso di scriverlo interamente in C++17 ponendo particolare attenzione alla leggibilità e all'efficienza. Di seguito ripercorro il ragionamento logico che mi ha guidato nella costruzione di questo modulo, passo dopo passo.

Formalizzazione Matematica del Dominio e del Grafo

Dal punto di vista matematico, il dominio spaziale continuo $\Omega = [0, 1]^2$ viene discretizzato in una griglia uniforme. Dato il parametro N , il passo spaziale risulta definito come $h = \frac{1}{N+1}$. L'insieme dei nodi interni, che corrispondono alle incognite numeriche del nostro problema differenziale, è descritto dall'insieme:

$$V = \{(x_i, y_j) \mid x_i = i \cdot h, y_j = j \cdot h \quad \forall i, j \in \{1, \dots, N\}\}$$

La cardinalità di tale insieme è $|V| = N^2$. Al fine di applicare lo stencil alle differenze finite a 5 punti, si costruisce un grafo non orientato $G = (V, E)$, in cui l'insieme degli spigoli E modella l'adiacenza fisica. Due nodi interni $u = (x_i, y_j)$ e $v = (x_k, y_l)$ sono tra loro adiacenti se e solo se la

loro distanza euclidea è esattamente pari al passo h :

$$E = \left\{ \{u, v\} \subset V \mid \sqrt{(x_i - x_k)^2 + (y_j - y_l)^2} = h \right\}$$

Operativamente, ciò implica che il nodo identificato dagli indici logici (i, j) è connesso ai suoi vicini orizzontali $(i \pm 1, j)$ e verticali $(i, j \pm 1)$, purché tali punti ricadano all'interno del dominio V . Agli spigoli viene associata una numerazione intera 1D (o ID), ottenuta concatenando le righe della griglia.

1. Impostazione e Controllo dei Parametri

Come prima cosa, avevo bisogno di impostare il punto di ingresso del mio programma e di assicurarmi di gestire correttamente la directory in cui avrei salvato i risultati. Ho quindi definito una costante logica per la cartella di destinazione e impostato un sistema robusto di controllo sui parametri: se il valore N passato da riga di comando è mancante o non valido (es. stringhe testuali o valori negativi), il programma non si interrompe bruscamente ma entra in un ciclo che richiede interattivamente all'utente un input corretto.

```
1 // Cartella in cui verranno salvati tutti i file generati
2 static const string CARTELLA_OUTPUT = "output/";
3
4 // Calcola l'identificatore numerico univoco (ID) di un nodo
5 inline int calcola_id_nodo(int i, int j, int N) {
6     return (i - 1) * N + (j - 1);
7 }
8
9 int main(int argc, char* argv[]) {
10     int N = 0;
11     bool valid = false;
12
13     if (argc == 2) {
14         try {
15             N = stoi(argv[1]);
16             if (N > 0) valid = true;
17             else cerr << "Errore: N deve essere > 0." << endl;
18         } catch (...) {
19             cerr << "Errore: intero non valido." << endl;
20         }
21     } else {
22         cerr << "Uso: " << argv[0] << " <N>" << endl;
23     }
24
25     while (!valid) {
26         cout << "Inserisci un valore intero N > 0: ";
27         string input;
28         if (!(cin >> input)) return 1;
29         try {
30             N = stoi(input);
31             if (N > 0) valid = true;
32             else cerr << "Errore: N deve essere > 0." << endl;
33         } catch (...) {
34             cerr << "Errore: formato non valido." << endl;
35         }
36     }
37
38     int numero_nodi = N * N;
39     // Crea la cartella output/ se non esiste
40     mkdir(CARTELLA_OUTPUT.c_str(), 0755);
```

2. Generazione dei Nodi e Scrittura delle Coordinate

Una volta preparato l'ambiente, il passo successivo è stato generare i nodi veri e propri. Ho scelto di scorrere la griglia iterando sulle righe e sulle colonne in modo naturale. Per ogni nodo

calcolo il suo ID univoco (tramite la funzione `calcola_id_nodo`) e determino le coordinate spaziali normalizzate nell'intervallo $(0, 1)$, scrivendo subito il risultato su disco per risparmiare memoria.

```
1 // --- Generazione Nodi ---
2 auto inizio_coordinate = high_resolution_clock::now();
3 ofstream file_coordinate(CARTELLA_OUTPUT + "coords.txt");
4 file_coordinate << fixed << setprecision(8);
5
6 for (int i = 1; i <= N; ++i) {
7     for (int j = 1; j <= N; ++j) {
8         int id_nodo = calcola_id_nodo(i, j, N);
9         double x = (double)i / (N + 1);
10        double y = (double)j / (N + 1);
11
12        // Formato: id riga colonna x y
13        file_coordinate << id_nodo << " " << i << " " << j
14            << " " << x << " " << y << "\n";
15    }
16 }
17 file_coordinate.close();
18 auto fine_coordinate = high_resolution_clock::now();
```

Ho forzato l'uso di `setprecision(8)` per essere sicuro che i valori in virgola mobile non perdessero precisione. Questo dettaglio si è rivelato fondamentale in seguito per calcolare in modo corretto la mediana geometrica nel modulo di partizionamento.

3. Costruzione del Grafo di Connessione

A questo punto avevo i nodi geometrici, ma mi serviva una struttura logica che rappresentasse i legami fisici della griglia. Ho perciò optato per una lista di adiacenza bidirezionale.

```
1 // --- Generazione Grafo di Connessione ---
2 auto inizio_connettivita = high_resolution_clock::now();
3 vector<vector<int>> lista_adiacenza(numero_nodi);
4 for (int i = 0; i < numero_nodi; ++i) {
5     lista_adiacenza[i].reserve(4); // Prevenzione riallocazioni
6 }
7
8 for (int i = 1; i <= N; ++i) {
9     for (int j = 1; j <= N; ++j) {
10        int u = calcola_id_nodo(i, j, N);
11        if (j < N) { // Connessione con il vicino in alto
12            int v_alto = calcola_id_nodo(i, j + 1, N);
13            lista_adiacenza[u].push_back(v_alto);
14            lista_adiacenza[v_alto].push_back(u);
15        }
16        if (i < N) { // Connessione con il vicino a destra
17            int v_destra = calcola_id_nodo(i + 1, j, N);
18            lista_adiacenza[u].push_back(v_destra);
19            lista_adiacenza[v_destra].push_back(u);
20        }
21    }
22 }
```

Per evitare che il vettore `lista_adiacenza` venisse continuamente ridimensionato (rallentando pesantemente l'esecuzione), ho pre-allocato la memoria usando `reserve(4)`, dal momento che sapevo in anticipo che i nodi avrebbero avuto al massimo 4 vicini. Nel ciclo principale, per evitare controlli ridondanti, ho scelto un approccio costruttivo “in avanti”: collego il nodo corrente soltanto con il suo vicino in alto e con quello a destra, aggiornando le due direzioni contemporaneamente.

4. Scrittura del Grafo ed Estrazione degli Archi

Infine, dovevo convertire questa struttura in memoria in un formato testuale semplice. Scorrendo la lista di adiacenza, ho imposto una basilare regola di disuguaglianza ($u < v$) in modo da essere certo di scrivere ogni arco non orientato esattamente una volta sola.

```
1      ofstream file_connettivita(CARTELLA_OUTPUT + "connectivity.txt");
2      for (int u = 0; u < numero_nodi; ++u) {
3          for (int v : lista_adiacenza[u]) {
4              if (u < v) { // Scrive l'arco non orientato una sola volta
5                  file_connettivita << u << " " << v << "\n";
6              }
7          }
8      }
9      file_connettivita.close();
10     auto fine_connettivita = high_resolution_clock::now();
```

5. Benchmark e Conclusione

Per accertarmi che le mie scelte implementative fossero davvero efficienti, ho integrato fin dall'inizio un sistema di *profiling* interno tramite la libreria `<chrono>`, per misurare i tempi effettivi di processamento.

```
1      // Benchmark delle performance temporali
2      duration<double, milli> tempo_coordinate =
3          fine_coordinate - inizio_coordinate;
4      duration<double, milli> tempo_connettivita =
5          fine_connettivita - inizio_connettivita;
6
7      cout << "Scrittura coords.txt      : "
8           << tempo_coordinate.count() << " ms\n";
9      cout << "Scrittura connectivity.txt : "
10          << tempo_connettivita.count() << " ms\n";
11
12     return 0;
13 }
```

Questa struttura dal design lineare mi ha fornito una base solida e scattante, regalandomi gli input puliti e minimali necessari per iniziare lo sviluppo del `task2_reorder.cpp`.

Modulo 2 — `task2_reorder.cpp`: nested dissection

Il Modulo 2 costituisce il cuore algoritmico del progetto. Il suo compito è calcolare una permutazione ottimizzata dei nodi del grafo tramite l'euristica della *nested dissection*, al fine di minimizzare il fill-in durante la fattorizzazione di Cholesky. Di seguito descrivo nel dettaglio le mie scelte architetturali e i passi implementativi.

Formalizzazione Matematica del Partizionamento

Da un punto di vista formale, l'algoritmo di nested dissection opera sul grafo $G = (V, E)$. L'obiettivo è individuare un separatore $V_S \subset V$ di cardinalità minima che disconnetta il grafo in due sottografi $G_1 = (V_1, E_1)$ e $G_2 = (V_2, E_2)$ approssimativamente della stessa dimensione, tali per cui non vi siano spigoli tra V_1 e V_2 . Dato un sottoinsieme di nodi $S \subseteq V$, la procedura seleziona iterativamente un asse di taglio (alternando x e y). Selezionato, ad esempio, l'asse x , si individua la coordinata mediana $\hat{x}$ e si definiscono:

$$V_1 = \{(x_i, y_j) \in S \mid x_i < \hat{x}\}$$

$$V_2 = \{(x_i, y_j) \in S \mid x_i > \hat{x}\}$$

$$V_S = \{(x_i, y_j) \in S \mid x_i = \hat{x}\}$$

I nodi in V_1 vengono enumerati per primi, seguiti dai nodi in V_2 , ed infine dai nodi del separatore V_S , applicando la medesima logica ricorsivamente su V_1 e V_2 .

1. Lettura dei Dati e Definizione delle Strutture Dati

Come prima operazione, avevo la necessità di caricare in memoria i dati geometrici generati dal Modulo 1. Poiché la logica si basa sulle sole coordinate, ho definito una struttura compatta `Node` e ho caricato l'intero contenuto del file `coords.txt` in un vettore. Ho poi implementato la lettura formale del file `connectivity.txt` per aderire ai requisiti rigorosi di progetto.

```
1 struct Node {
2     int id;
3     int i;
4     int j;
5     double x;
6     double y;
7 };
8
9 // ...
10 vector<Node> nodes;
11 int n, i, j;
12 double x, y;
13 while (coords_file >> n >> i >> j >> x >> y) {
14     nodes.push_back({n, i, j, x, y});
15 }
```

2. Implementazione Iterativa con Stack Esplicito

L'algoritmo di nested dissection ha intrinsecamente una natura ricorsiva. Tuttavia, per griglie molto grandi ($N \gg 1000$), un approccio basato sulle normali chiamate a funzione rischiava di esaurire lo spazio dello stack, provocando crash indesiderati (*stack overflow*). Ho deciso quindi di convertire la logica in una forma puramente iterativa, impiegando uno `std::stack` esplicito allocato nell'heap, che mi ha garantito grande stabilità.

```
1 struct StackItem {
2     vector<Node> S;    // sottoinsieme di nodi da processare
3     int axis;          // asse di taglio corrente (0=X, 1=Y)
4     bool separatore;   // flag: indica se l'item e' un separatore finale
5 };
6
7 void nested_dissection(const vector<Node>& nodi_iniziali, int
8     axis_iniziale, vector<int>& ordering) {
9     stack<StackItem> stk;
10    stk.push({nodi_iniziali, axis_iniziale, false});
11
12    while (!stk.empty()) {
13        StackItem item = move(stk.top());
14        stk.pop();
15        // ...
16    }
```

3. Estrazione del Separatore Geometrico

All'interno del ciclo principale, procedo all'estrazione delle coordinate minime e massime per calcolare la mediana. Una volta determinato il valore medio, suddivido i nodi nei tre insiemi V_1, V_2, V_3 . La vera accortezza risiede nell'ordine di inserimento nello stack: dovendo processare per primo l'insieme V_1 , esso deve essere inserito per ultimo (*Last-In, First-Out*).

```

1      // Calcolo coordinata mediana
2      int mid_val = (min_val + max_val) / 2;
3
4      vector<Node> V1, V2, VS;
5      for (const auto& node : item.S) {
6          int val = (item.axis == 0) ? node.i : node.j;
7          if (val < mid_val)      V1.push_back(node);
8          else if (val > mid_val) V2.push_back(node);
9          else                    VS.push_back(node);
10     }
11
12     // Push invertito per eseguire V1 -> V2 -> VS
13     stk.push({VS, item.axis,      true});
14     stk.push({V2, 1 - item.axis,  false});
15     stk.push({V1, 1 - item.axis,  false});
16 }
17 }

```

4. Salvataggio della Permutazione e Benchmark

Al termine del processo iterativo, l'ordinamento è pronto e viene salvato sul vettore. Per calcolare il tempo di esecuzione, ho utilizzato la libreria `<chrono>`, ripetendo un pattern di misurazione simile a quello che ho impiegato nel Modulo 1. L'output finale viene salvato nel file `ordering.txt` secondo un formato che fa corrispondere la nuova posizione (l'indice) al vecchio identificatore logico del nodo.

```

1      // ... avvio timer chrono ...
2      nested_dissection(nodes, 0, ordering);
3      // ... arresto timer chrono ...
4
5      ofstream out_file(OUTPUT_DIR + "ordering.txt");
6      for (size_t k = 0; k < ordering.size(); ++k) {
7          out_file << k << " " << ordering[k] << "\n";
8      }
9      out_file.close();

```

Questo secondo modulo risulta così essere robusto da un punto di vista algoritmico, limitando l'uso della memoria e offrendo un tempo asintotico di esecuzione atteso pari a $O(N^2 \log N)$.

Modulo 3 — `task3_assemble.cpp`: assemblaggio del sistema lineare

Il Modulo 3 è incaricato di costruire il sistema lineare che approssima l'equazione differenziale. Ho progettato questo task per accettare sia l'ordinamento naturale della griglia, sia quello ottimizzato prodotto dal Modulo 2 tramite l'apposito flag `-reorder`.

Formalizzazione Matematica del Sistema Lineare

Dal punto di vista matematico, lo scopo è assemblare il sistema $Au = b$. L'equazione del calore stazionaria, $\kappa \nabla^2 u + f = 0$, viene approssimata utilizzando lo stencil alle differenze finite a 5 punti. Per ogni nodo k con coordinate $(x_{i(k)}, y_{j(k)})$ e considerando il passo spaziale $h = \frac{1}{N+1}$, i coefficienti

della matrice sparsa A risultano:

$$\begin{aligned} A_{k,k} &= -\frac{4\kappa}{h^2} \\ A_{k,k'} &= +\frac{\kappa}{h^2} \quad \text{per ogni nodo interno adiacente } k' \end{aligned}$$

Il termine noto viene posto pari a $b_k = -f(x_{i(k)}, y_{j(k)})$. Nel caso in cui il nodo adiacente k' cada sul bordo del dominio, il corrispondente valore è fissato a zero (condizioni di Dirichlet omogenee), perciò il suo contributo viene spostato al termine noto b_k che, in questo caso specifico, non subisce alterazioni numeriche.

1. Gestione degli Argomenti e Lettura Coordinate

Per garantire robustezza, la prima operazione che eseguo è il controllo degli argomenti da riga di comando per l'attivazione opzionale della nested dissection. Successivamente, leggo il file `coords.txt`, allocando uno spazio predefinito per le coordinate dei nodi e assicurandomi che il numero di entrate coincida con N^2 .

```
1  int N = stoi(argv[1]);
2  bool usa_reorder = false;
3  if (argc == 3 && string(argv[2]) == "--reorder") {
4      usa_reorder = true;
5  }
6
7  // ... Lettura di coords.txt ...
8  while (file_coords >> id >> ri >> rj >> rx >> ry) {
9      coords[id] = {id, ri, rj, rx, ry};
10     contatore_nodi++;
11 }
```

2. Lettura della Permutazione (Opzionale)

Se richiesto, carico il file `ordering.txt` e costruisco le mappe di permutazione diretta e inversa, fondamentali per mappare la topologia logica del problema fisico all'interno della struttura riordinata della matrice in tempo costante $O(1)$. Se l'ordinamento richiesto è quello naturale, istanzio semplicemente un vettore identità.

```
1  vector<int> perm(num_nodi);
2  vector<int> inv_perm(num_nodi);
3
4  if (usa_reorder) {
5      // ... Lettura new_id e old_id da file ...
6      perm[new_id] = old_id;
7      inv_perm[old_id] = new_id;
8  } else {
9      for (int k = 0; k < num_nodi; ++k) {
10         perm[k] = k;
11         inv_perm[k] = k;
12     }
13 }
```

3. Assemblaggio dei Coefficienti dello Stencil

La fase di assemblaggio vera e propria itera sui nodi in funzione del loro nuovo indice k . Per ogni nodo calcolo l'entrata diagonale e applico lo stencil esplorando i 4 vicini topologici (sinistra, destra, basso, alto). Nel calcolo uso i coefficienti costanti pre-calcolati dipendenti da κ e h . Per minimizzare le allocazioni, memorizzo le entrate non nulle in un vettore di `Triplet` pre-allocato (`reserve`).

```
1  for (int k = 0; k < num_nodi; ++k) {
2      int old_id = perm[k];
3      // ... estrazione coordinate (ri, rj) ...
4
5      triplets.push_back({k, k, coeff_diag});
6      rhs[k] = -f_sorgente(x, y);
7  }
```

```

8         for (int d = 0; d < 4; ++d) {
9             int ni = ri + di[d];
10            int nj = rj + dj[d];
11
12            if (ni < 1 || ni > N || nj < 1 || nj > N) {
13                // Condizione di Dirichlet sul bordo
14                rhs[k] -= coeff_fuori_diag * 0.0;
15            } else {
16                // Nodo interno riordinato
17                int old_id_vicino = id_naturale(ni, nj, N);
18                int k_vicino      = inv_perm[old_id_vicino];
19                triplets.push_back({k, k_vicino, coeff_fuori_diag});
20            }
21        }
22    }

```

4. Salvataggio della Matrice sparsa e del Termine Noto

Infine, salvo i risultati esportando il vettore termine noto e i tripli iterando linearmente sulla collezione. Ho applicato la modalità di scrittura formattata `fixed` con precisione decimale di 10 cifre per assicurare la conservazione dei coefficienti numerici in virgola mobile ad alta precisione. L'algoritmo assembla il tutto in un tempo di elaborazione lineare $O(N^2)$, un aspetto critico per scalare facilmente il codice a reti con un gran numero di incognite.

Modulo 4 — `solver.ipynb`: soluzione e analisi

Input.

- `A.txt` e `rhs.txt` in formato sparso (i, j, val) / vettore.
- Parametro N (o serie di valori $N \in \{32, 64, 128, 256, 512, 1024\}$).

Output.

- Soluzione $u \in \mathbb{R}^{N^2}$ del sistema $Au = b$ (riordinata nella griglia originale per la visualizzazione).
- Spy plot della struttura di A prima e dopo il riordinamento.
- Grafico della soluzione $u(x, y)$ come superficie o heatmap 2D.
- Tabella/grafico comparativo di tempi di fattorizzazione e numero di elementi non nulli di L per le due strategie di ordinamento.

Funzionalità richieste.

- Lettura dei file e costruzione di `scipy.sparse.csc_matrix` (o `coo_matrix`), che gestiscono automaticamente la somma delle entrate duplicate dovute alla simmetria del file `A.txt`.
- Fattorizzazione $-A = LL^T$ tramite `sksparse.cholmod.cholesky` con `order="natural"` (l'ordinamento è già codificato in A).
- Soluzione del sistema lineare tramite `scipy.sparse.linalg.spsolve_triangular`.
- Misurazione del tempo di esecuzione (con `time.perf_counter`) per ciascun valore di N e per entrambe le strategie di ordinamento.
- Confronto del fill-in: conteggio di `nnz(L)` per ordinamento naturale e nested dissection.

Strutture dati.

- `scipy.sparse.csc_matrix` per A (e $-A$ per la fattorizzazione).
- Array `numpy.ndarray` per il vettore b e la soluzione u .
- Oggetto `Factor` di `CHOLMOD` per la fattorizzazione.

Complessità attesa. Con ordinamento naturale: fill-in $O(N^3)$ e risoluzione $O(N^3)$. Con nested dissection: fill-in $O(N^2 \log N)$ e risoluzione $O(N^2 \log N)$.

Dipendenze. Legge `A.txt` e `rhs.txt` prodotti dal Modulo 3; non genera file ma produce output grafico nel notebook.

4 Analisi sperimentale attesa

L'obiettivo dell'analisi sperimentale è quantificare il vantaggio computazionale della nested dissection rispetto all'ordinamento naturale. Si prevedono le seguenti misurazioni per $N \in \{32, 64, 128, 256, 512, 1024\}$:

Grandezza misurata	Ordinamento naturale	Nested dissection
<code>nnz(A)</code>	$O(N^2)$	$O(N^2)$
<code>nnz(L)</code> (fill-in)	$O(N^3)$	$O(N^2 \log N)$
Tempo di fattorizzazione	$O(N^3)$	$O(N^2 \log N)$ (atteso)
Tempo di risoluzione	$O(N^3)$	$O(N^2 \log N)$ (atteso)

Il notebook produrrà:

1. Spy plot di A e di L (fattore di Cholesky) per entrambi gli ordinamenti, per evidenziare visivamente la differenza di fill-in.
2. Heatmap 2D o superficie 3D della soluzione $u(x, y)$ per almeno un valore di N .
3. Grafico log-log di `nnz(L)` in funzione di N : le due curve dovrebbero separare nettamente attorno a $N \geq 64$. Verrà esplicitamente calcolato anche il rapporto $\text{nnz}(L_{\text{natural}})/\text{nnz}(L_{\text{ND}})$ (fill-in relativo) per evidenziare il risparmio di memoria.
4. Grafico dei tempi di fattorizzazione in funzione di N , con indicazione della pendenza empirica per stimare l'esponente di crescita.